A COMPLETE VISUAL TUTORIAL

Naive Bayes
Hyperparameter Tuning

From the very basics to pro-level tuning with GridSearchCV

You will learn	Covered
What Naive Bayes is and why it is "naive"	Bayes' theorem, priors, likelihoods, worked example
Every hyperparameter	alpha, fit_prior, binarize, var_smoothing
The math behind each one	Smoothing formulas, Gaussian curves, log-space
Five optimizations	Log space, correlated features, pre-processing, normalizing, parallelizing
Real code	Grid dictionaries and a full GridSearchCV template, explained line by line

Built from a video script, reorganised into a step-by-step tutorial with diagrams, formulas and examples. Nothing from the original content was dropped.

Roadmap

This tutorial climbs in levels. Read in order the first time; later you can jump straight to a part as a reference.

Level	Part	What you get
Beginner	1. The idea	Bayes' theorem, why "naive", a spam example with real numbers
Beginner	2. What is a hyperparameter	Parameters vs hyperparameters, the four Naive Bayes variants
Intermediate	3. alpha (smoothing)	The zero-frequency trap, the formula, bias-variance, grid values
Intermediate	4. fit_prior and binarize	Class probabilities and thresholding
Intermediate	5. var_smoothing	Gaussian Naive Bayes and bell curves
Advanced	6. Five optimizations	Log space, correlated features, stop-words, normalize, parallelize
Pro	7. Grid Search in practice	Ready-to-use grids, full template, line-by-line, cheat sheet

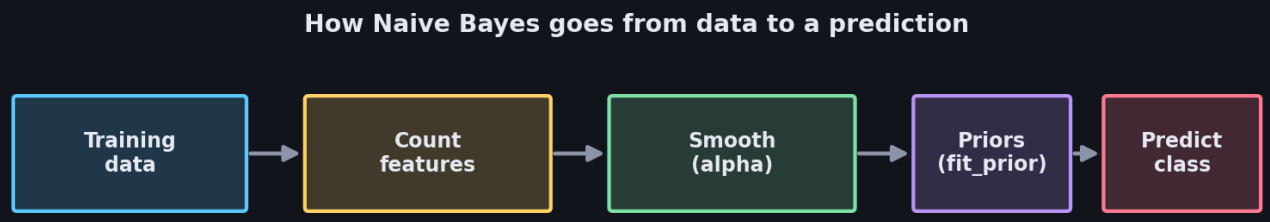


Figure 1. The big picture: every hyperparameter in this tutorial sits somewhere in this pipeline.

PART 1

The Idea Behind Naive Bayes

Beginner level: no prior knowledge needed

What problem does it solve?

Naive Bayes is a **classification** algorithm. Given some evidence (for example the words in an email), it answers: *which class is most likely?* (Spam or Not Spam, Positive review or Negative review). It is called **intentionally simple and fast**: it has very few hyperparameters to tune compared with models like Decision Trees or Support Vector Machines. That is exactly why it is a favourite for text classification.

Bayes' theorem in plain English

Bayes' theorem lets you flip a probability around. You usually know "*how often do spam emails contain the word 'free'?*" but you want to know "*this email contains 'free', so how likely is it spam?*"

BAYES' THEOREM

$$P(class \mid words) = P(words \mid class) \times P(class) / P(words)$$

Term	Name	Plain-English meaning
P(class | words)	Posterior	What we want: probability of the class after seeing the words
P(words | class)	Likelihood	How typical these words are for that class
P(class)	Prior	How common the class is before seeing any words (baseline)
P(words)	Evidence	Same for every class, so we can ignore it when comparing classes

Why is it called "naive"?

A real email has many words, and words depend on each other ("free" and "money" often appear together). Calculating all those dependencies is very hard. Naive Bayes **naively assumes every feature is independent** given the class. That lets us simply multiply the individual probabilities:

THE NAIVE ASSUMPTION (∝ means "proportional to")

$$P(class \mid x_1,...,x_n) \propto P(class) \times P(x_1|class) \times P(x_2|class) \times ... \times P(x_n|class)$$

Worked example: a tiny spam filter

Suppose training data says 30% of emails are Spam and 70% are Not Spam. Word likelihoods learned from the data:

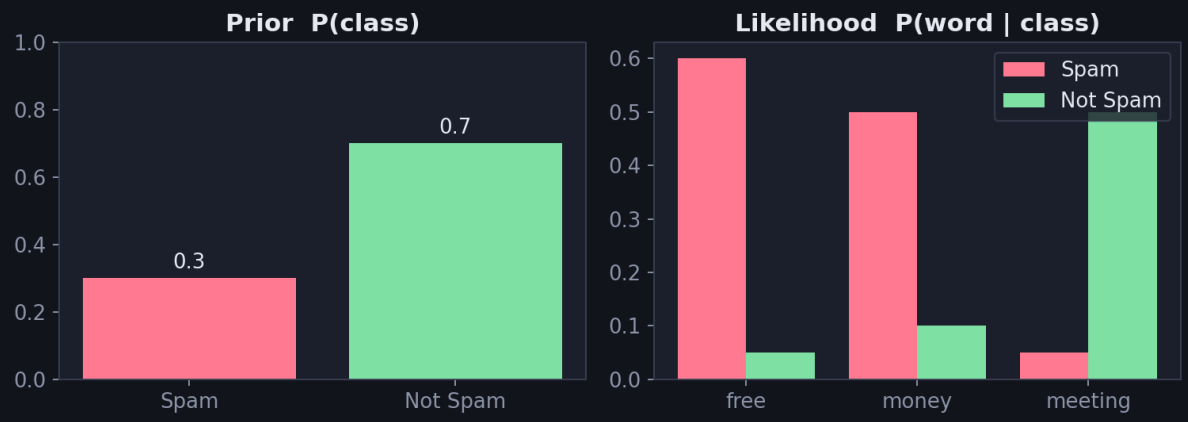


Figure 2. Left: priors. Right: how likely each word is inside each class.

Word	P(word | Spam)	P(word | Not Spam)
free	0.60	0.05
money	0.50	0.10
meeting	0.05	0.50

A new email contains **"free"** and **"money"**. Compute a score for each class:

Spam score = 0.30 × 0.60 × 0.50 = **0.0900**

Not Spam score = 0.70 × 0.05 × 0.10 = **0.0035**

Divide each by the total (0.0900 + 0.0035 = 0.0935) to turn scores into probabilities:

RESULT

$$P(\text{Spam} \mid \text{email}) = 0.0900 / 0.0935 \approx 96.3\%$$

$$P(\text{Not Spam} \mid \text{email}) = 0.0035 / 0.0935 \approx 3.7\%$$

Key takeaway
Naive Bayes = **prior** × a **chain of multiplied likelihoods**. Everything we tune later exists to make that multiplication chain safe, stable and fair.

PART 2

Hyperparameters and the Four Variants

Beginner to intermediate

Parameter vs hyperparameter

Parameters are learned automatically from data (for example the word probabilities). **Hyperparameters** are settings *you* choose before training (for example `alpha`). Tuning means trying several hyperparameter values and keeping the best one, which is what a Grid Search does.

The hyperparameters you can tune depend on the exact variation of Naive Bayes you are using (typically in Scikit-Learn). Pick the variation by your data type:

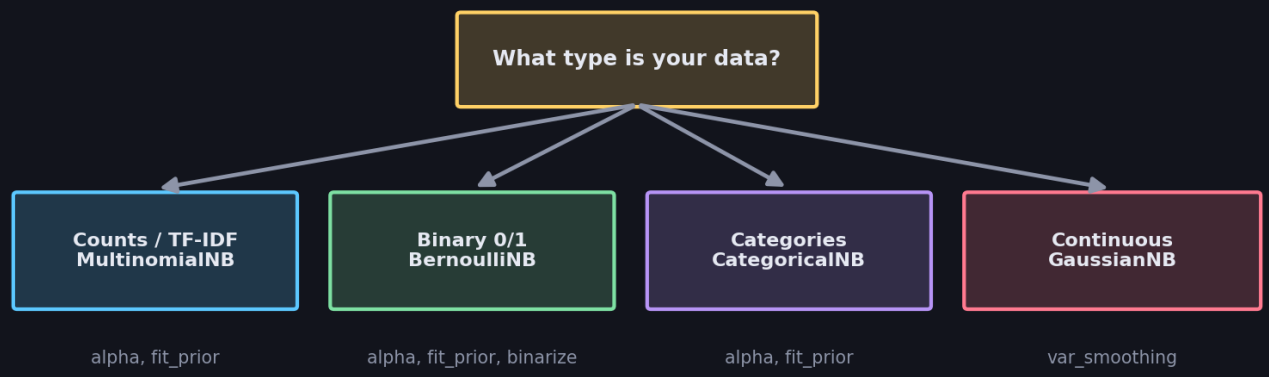


Figure 3. Which Naive Bayes variant fits your data, and which hyperparameters it has.

Variant	Best for	Hyperparameters
MultinomialNB	Word counts, text, discrete counts (the standard choice for text)	alpha, fit_prior
BernoulliNB	Strictly binary data: word present (1) or absent (0)	alpha, fit_prior, binarize
CategoricalNB	Features that are categories (colour, city, grade)	alpha, fit_prior
GaussianNB	Continuous numbers (age, square footage) assumed bell-curve shaped	var_smoothing

Summary of the four hyperparameters

Hyperparameter	Applies to	Plain-English job	Grid values from the script
alpha	Multinomial, Bernoulli, Categorical	Smoothing: stops any probability being exactly 0	[0.01, 0.1, 0.5, 1.0, 10.0]
fit_prior	Multinomial, Bernoulli, Categorical	Learn class probabilities from data, or assume equal	[True, False]

Hyperparameter	Applies to	Plain-English job	Grid values from the script
binarize	BernoulliNB	Threshold that converts numbers to 0 or 1	[None, 0.0, 0.5, 1.0]
var_smoothing	GaussianNB	Adds tiny variance so curves never collapse	[1e-9, 1e-8, 1e-7, 1e-6, 1e-5]

PART 3

alpha: Additive Smoothing

Intermediate: the most important hyperparameter

In Naive Bayes (specifically the Multinomial, Bernoulli and Categorical variations), **alpha is by far the most frequently tuned hyperparameter**. It dictates how the model handles unseen data and directly controls the bias-variance trade-off.

The problem: the "zero-frequency" trap

Naive Bayes calculates conditional probabilities from feature frequencies in training data, then **multiplies** them. Imagine a spam filter. During training the word **"crypto"** appears only in Spam emails and never in Not Spam. A new, completely legitimate email mentions "crypto". The model assigns **exactly 0%** to the Not Spam category for that word. Because everything is multiplied, one zero wipes out the whole equation. The model can mathematically *never* classify that email as Not Spam, no matter how many other safe words it contains.

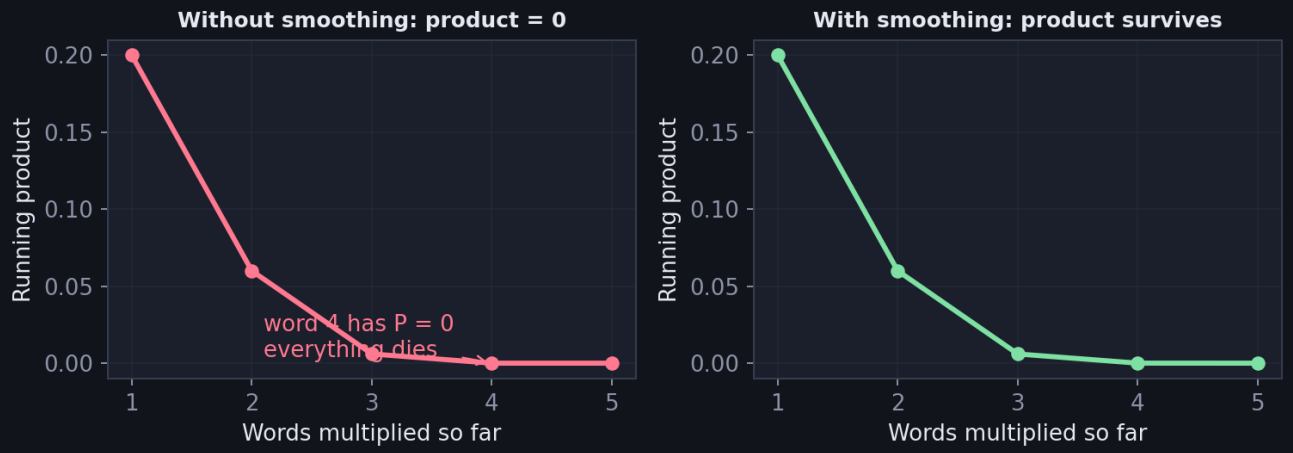


Figure 4. Left: a single zero kills the running product. Right: with a tiny non-zero value the chain survives.

WITHOUT SMOOTHING

$$P(\text{Not Spam} \mid \text{email}) = 0.7 \times 0.2 \times 0.3 \times 0.1 \times 0 \times 0.4 = 0$$

The fix: additive smoothing

alpha introduces **Additive Smoothing**. It adds a fake, artificial count (the alpha value) to every single feature. Names to remember:

- **Laplace smoothing**: alpha = 1.0
- **Lidstone smoothing**: alpha < 1.0 (for example 0.1 or 0.01)

With alpha = 1.0 the model pretends it saw the word "crypto" in the Not Spam category at least one time. No probability is ever exactly zero, so the multiplication chain survives and the model can evaluate the whole email fairly.

THE SMOOTHING FORMULA (Multinomial)

$$P(\text{word}_i \mid \text{class}) = \left(\text{count}_{i,\text{class}} + \alpha \right) / \left(\text{total}_{\text{class}} + \alpha \times V \right)$$

Symbol	Meaning
count(i,class)	How many times word i appeared in documents of that class
total(class)	Total number of word occurrences in that class
α (alpha)	The fake count you add to every word
V	Vocabulary size (number of distinct words). Added α for each word, so the denominator grows by α × V and probabilities still sum to 1

Other variants use the same idea
BernoulliNB: $P(x_i=1 \mid class) = (N_{i,class} + \alpha) / (N_{class} + 2\alpha)$, because each feature has 2 outcomes (0 or 1).
CategoricalNB: $(N_{i,c} + \alpha) / (N_{class} + \alpha \times n_{categories})$, because each feature has n possible categories.

Worked numbers

Say the Not Spam class has total = 100 word occurrences and vocabulary V = 50. "crypto" appeared 0 times; a common word appeared 30 times.

alpha	P(crypto | Not Spam) count = 0	P(common word) count = 30	What it means
0 (none)	0 / 100 = 0.0000	30 / 100 = 0.3000	Zero trap!
0.01	0.01/100.5 = 0.0001	30.01/100.5 = 0.2986	Barely changes the data
0.1	0.1/105 = 0.0010	30.1/105 = 0.2867	Gentle smoothing
1.0 (Laplace)	1/150 = 0.0067	31/150 = 0.2067	Standard default
10	10/600 = 0.0167	40/600 = 0.0667	Differences shrinking
100	100/5100 = 0.0196	130/5100 = 0.0255	Rare ≈ common: pattern lost

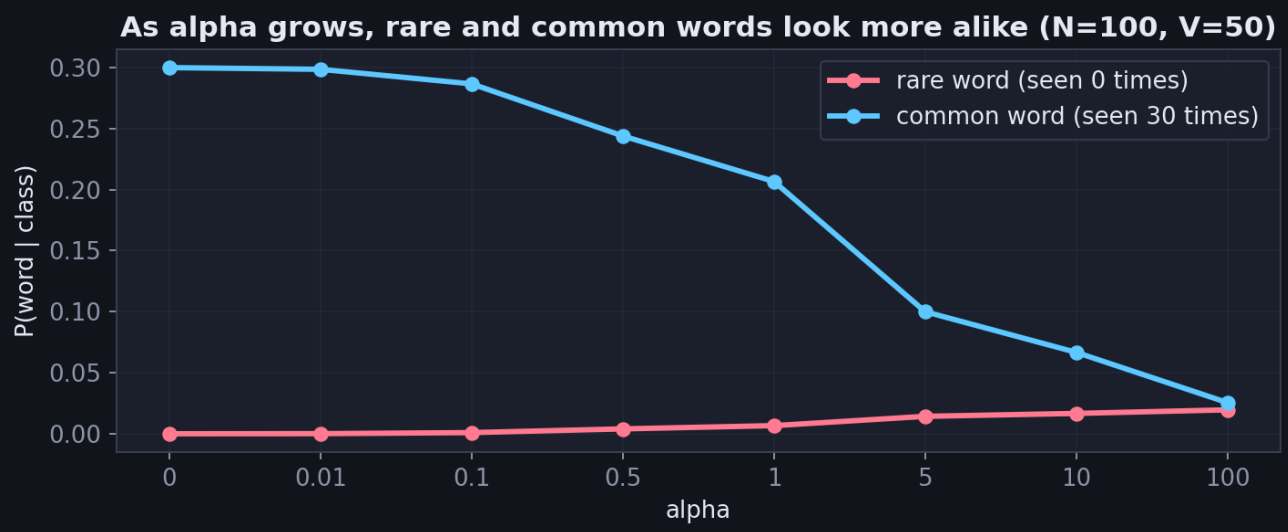


Figure 5. Watch the two lines converge as alpha grows: the model can no longer tell a rare word from a common one.

How alpha affects the model (bias vs variance)

Adjusting alpha directly dictates how strictly the model adheres to your training data:

- **Very low alpha (e.g. 0.001):** barely alters the raw data. The model stays highly sensitive to the exact frequencies in the training set. This can lead to **overfitting (high variance)** because it trusts the training data too much.
- **High alpha (e.g. 10 or 100):** adds massive artificial counts to everything. This smooths out differences between probabilities, making all features seem more equally distributed. This leads to **underfitting (high bias)** because the model loses the ability to distinguish strong patterns from weak ones.

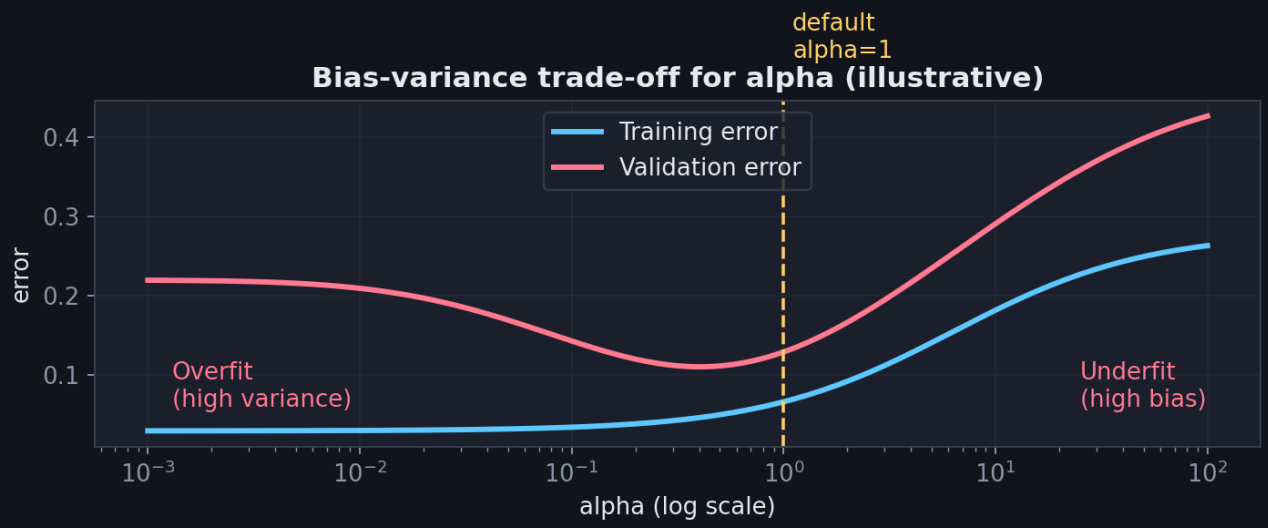


Figure 6. The sweet spot sits in the middle; Grid Search finds it for you.

Values to try in Grid Search

Because the effect of alpha scales exponentially, test values across a **logarithmic scale**, not a linear one (0.1, 0.2, 0.3 would tell you almost nothing; 0.01, 0.1, 1, 10 tells you a lot).

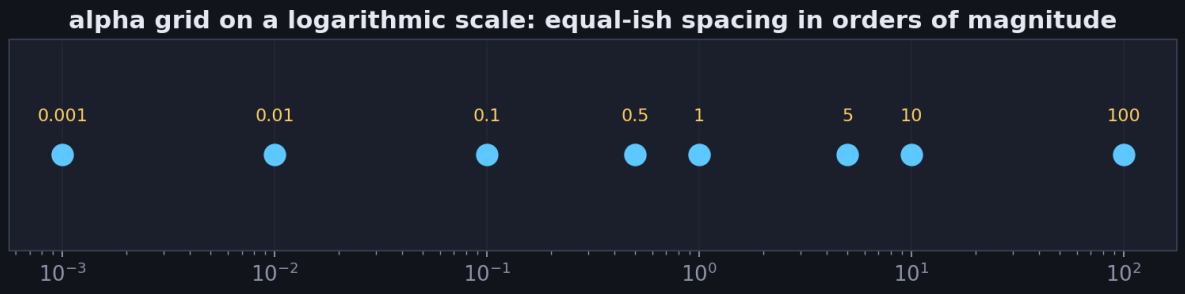


Figure 7. Log-scale spacing for alpha.

Grid	Used when
[0.01, 0.1, 0.5, 1.0, 10.0]	Quick first search (the short version, 1.0 = standard Laplace; below 1.0 = Lidstone)
[0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0, 100.0]	Wide, thorough search across the full logarithmic range

Tuning strategy
Standard default: 1.0 is the default in Scikit-Learn and a highly stable baseline.
Massive dataset with rich vocabulary: smaller alpha (0.1 or 0.01) usually performs better.
Very small dataset: a higher alpha prevents the model from jumping to conclusions.

Good to know (added tip)
Scikit-Learn quietly raises alpha values smaller than about 1e-10 to 1e-10 to avoid numerical trouble, so do not try alpha = 0 expecting "no smoothing".

PART 4

fit_prior and binarize

Intermediate: class probabilities and thresholds

fit_prior: class probabilities

Applies to: MultinomialNB, BernoulliNB, CategoricalNB.

This tells the model whether it should calculate the baseline probability of each class directly from your training data. If 80% of your dataset is Class A and 20% is Class B:

- `fit_prior=True`: the model learns the 80/20 split and assumes Class A is generally more likely.
- `fit_prior=False`: the model ignores your dataset's imbalance and assumes all classes are equally likely (50/50).

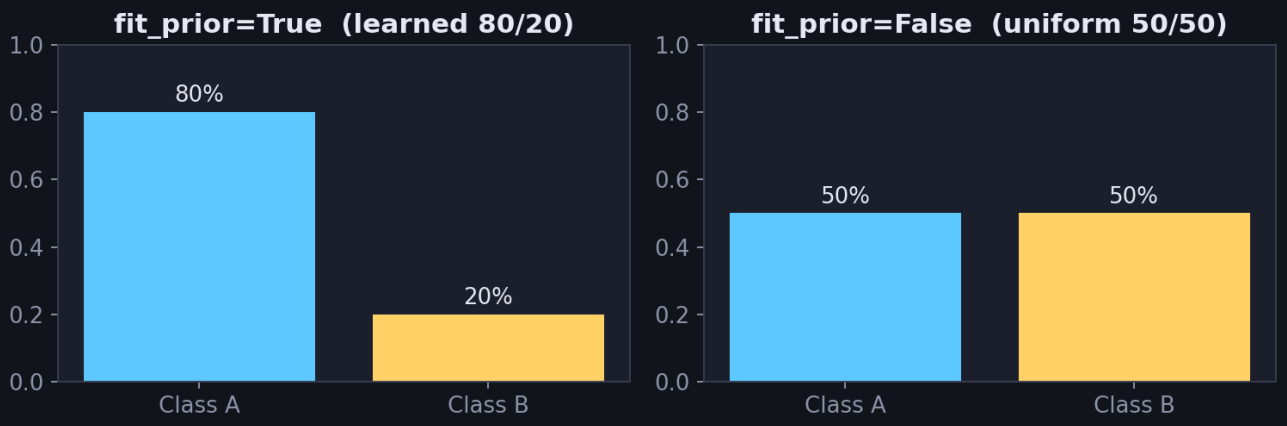


Figure 8. Learned prior vs uniform prior.

THE MATH

fit_prior=True: P(class) = (docs in class) / (all docs)

fit_prior=False: P(class) = 1 / (number of classes)

When to use which
Use True when your training data reflects reality (e.g. truly 80% of real emails are not spam). Try False when the data is imbalanced only because of how you collected it, or when you do not want the model to favour the majority class. Grid Search values: `[True, False]`. For heavily imbalanced classes, also swap the scoring metric to `f1_macro` (see Part 7).

binarize: thresholding

Applies to: BernoulliNB only.

Bernoulli Naive Bayes expects **strictly binary data** (0s and 1s), such as "word present / not present". If your dataset has continuous numbers or frequency counts, `binarize` sets a threshold to convert everything to 0 or 1: anything above the threshold becomes 1, everything else becomes 0. If your data is already 0s and 1s, set it to None.

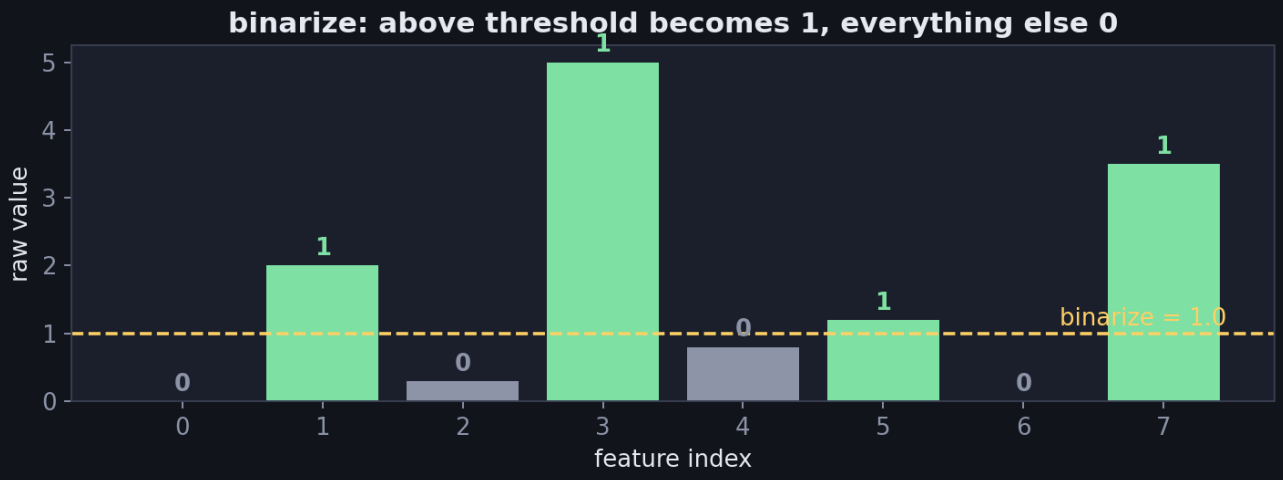


Figure 9. With binarize = 1.0, only values strictly above 1.0 turn into 1.

Value	Effect
None	Data is already 0/1; leave it untouched
0.0	Anything greater than 0 becomes 1 ("present at all"): common for word counts
0.5	Threshold in the middle of a 0 to 1 scale
1.0	Needs more than 1 occurrence to count as present

Grid Search values: `[None, 0.0, 0.5, 1.0]`. Adjust these numbers based on the scale of your raw data. (The ready-made BernoulliNB grid in Part 7 uses `[None, 0.0, 0.5]`.)

PART 5

var_smoothing: The Continuous Counterpart

Intermediate: for GaussianNB

If you use Gaussian Naive Bayes to evaluate continuous numbers (like predicting house prices from exact square footage or age), **alpha does not exist**. The primary hyperparameter to tune instead is `var_smoothing`.

Background: the Gaussian assumption

GaussianNB assumes each continuous feature follows a normal (bell curve) distribution within every class. For each class it learns a mean μ and a variance σ² per feature, and scores a new value x with the bell curve:

GAUSSIAN PROBABILITY DENSITY

$$P(x \mid class) = 1 / \sqrt{(2\pi\ \sigma^2)} \times \exp(-(x - \mu)^2 / (2\sigma^2))$$

The problem it solves

If a feature has almost zero variance (for example all houses in the dataset were built in the exact same year), the bell curve becomes **infinitely narrow**. Look at the formula: σ² sits in a denominator, so σ² → 0 causes **division-by-zero errors** when calculating probabilities for new data points.

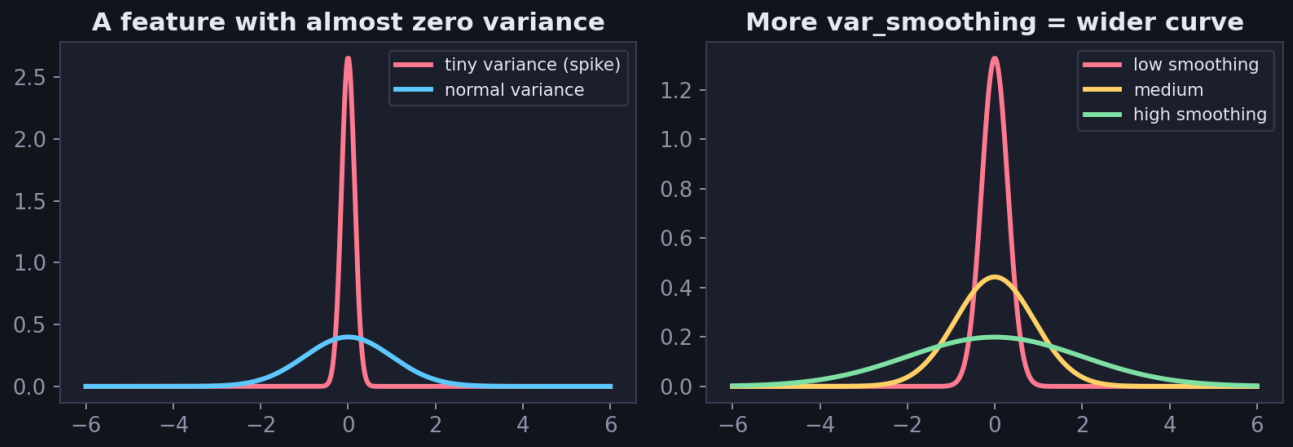


Figure 10. Left: a near-zero variance spike. Right: wider curves as var_smoothing grows.

How it works

`var_smoothing` takes the feature with the **absolute largest variance** in your entire dataset, multiplies that large variance by a microscopic fraction (the var_smoothing value), and adds that chunk of variance to **every** feature's curve. Every bell curve widens slightly, so none collapses into a single vertical line.

THE MATH

$$\varepsilon = \text{var_smoothing} \times \max(\text{Var}(x_1), \ldots, \text{Var}(x_n))$$

$$\sigma^2_j \text{ (new)} = \sigma^2_j \text{ (old)} + \varepsilon$$

Example
Largest variance in your data = 400 and var_smoothing = 1e-9. Then ε = 400 × 0.000000001 = 0.0000004. A feature with variance 0 becomes 0.0000004, so the division is safe, yet the curve is still extremely narrow.

How it affects the model

- **Low var_smoothing:** bell curves stay narrow and highly fitted to the training data. Risks overfitting and mathematical instability.
- **High var_smoothing:** curves become wider and flatter. This blurs the distinction between classes, increasing bias and risking underfitting.

Values to try in Grid Search

You add only a tiny fraction of variance, so test exceptionally small numbers on a logarithmic scale.

Grid	Notes
[1e-9, 1e-8, 1e-7, 1e-6, 1e-5]	Short, practical grid
[1e-11, 1e-10, 1e-9, 1e-8, 1e-7, 1e-5]	Wider grid with extra-small values
np.logspace(0, -9, num=10)	Auto-generates 10 values: 1, 0.1, 0.01, ..., 1e-9 (see note)

Standard default and a note
1e-9 is the standard baseline in Scikit-Learn. The logspace grid starts at 1 (which adds as much variance as the largest feature has), so the top values are very aggressive. If results look flat or poor, narrow it to something like `np.logspace(-2, -11, num=10)` (added tip).

PART 6

Five Optimizations

Advanced: fixing the algorithm's natural blind spots

Think of these five optimizations as fixing Naive Bayes' natural blind spots. They become especially obvious with text data such as movie reviews.

1. Stop multiplying tiny numbers (Log Space)

When scoring a long review, the model multiplies hundreds of tiny fractions (like 0.01 × 0.005 × 0.02). The final number becomes so microscopic that the computer's memory gives up, rounds it to **exactly zero**, and ruins the prediction (this is called *numerical underflow*). Logarithms change the math from **multiplying tiny decimals to adding normal numbers**, completely preventing the glitch.

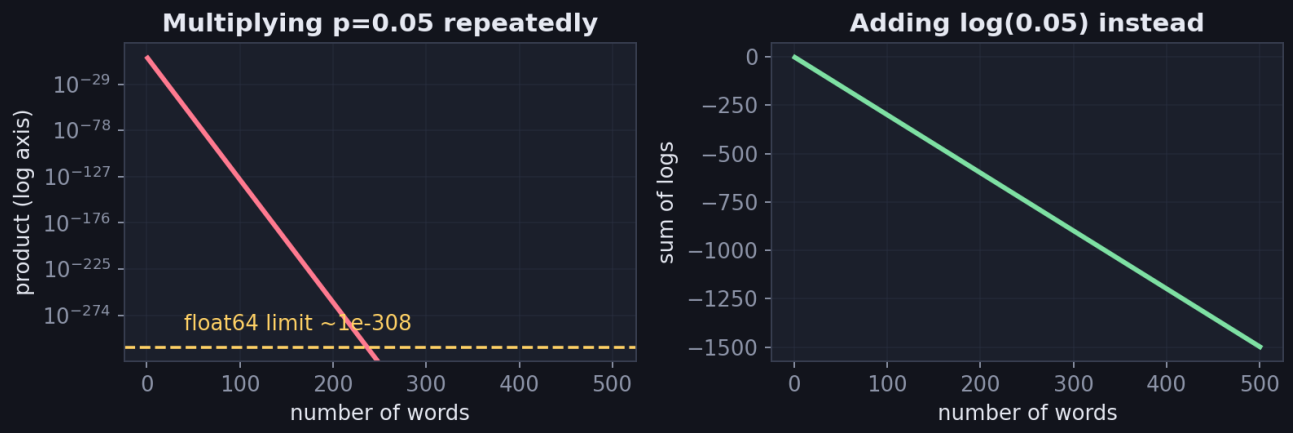


Figure 11. Left: the product races past the smallest number a computer can store. Right: the same information as a harmless sum.

LOG-SPACE FORM (log of a product = sum of logs)

$$\log P(\text{class} \mid x) \propto \log P(\text{class}) + \Sigma \log P(x_i \mid \text{class})$$

Good news
Scikit-Learn's Naive Bayes already works in log space internally. You mainly need this idea when writing Naive Bayes by hand, and it explains why predict_log_proba exists.

2. Don't double-count clues (Correlated Features)

The algorithm is "naive" because it assumes every feature is independent. If two columns essentially measure the exact same thing (for example height in cm and height in inches), the model blindly treats them as two separate, independent proofs and **unfairly double-counts** their importance.

- **Fix:** remove or merge near-duplicate columns before training.
- Check a correlation matrix and drop one of any pair with very high correlation.

3. Cut the dead weight (Text Pre-processing)

Words like "the", "watch" or "film" appear constantly in both 1-star and 5-star ratings. They carry no actual sentiment; they only give the computer more useless math. **Deleting them** (stop-word and domain-word removal) makes the model leaner and faster.

4. Level the playing field (Normalize)

A quick one-sentence review should not be mathematically overpowered by a massive 10-paragraph critique just because of word counts. Normalizing turns raw counts into **proportions** (e.g. "what percentage of this review is the word 'masterpiece'?"), so short and long texts are judged on the same scale.

NORMALIZED TERM FREQUENCY

```
tf(word, review) = count(word) / total words in review
```

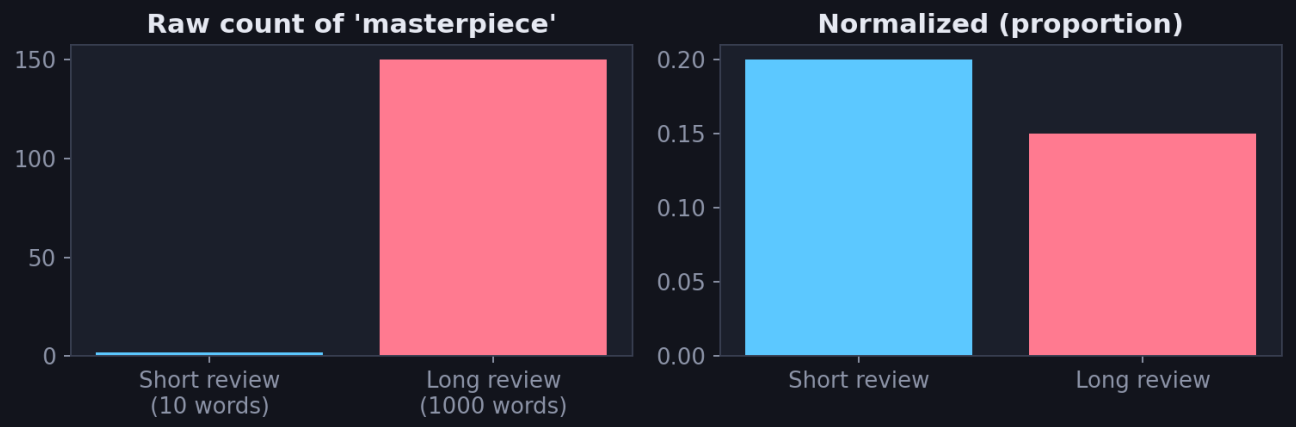


Figure 12. The long review has more raw counts (150 vs 2), but the short review actually uses the word more (20% vs 15%).

5. Divide and conquer (Parallelize)

Because the model evaluates every feature independently, it never has to wait for one calculation to finish before starting the next. You can split the work across multiple CPU cores and process a massive dataset at the same time. In Scikit-Learn this is the `n_jobs=-1` option in GridSearchCV (Part 7).

Pro-level preprocessing snippet (added example)

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", norm="l2")),  # #3 stop-words, #4 normalize
    ("nb",    MultinomialNB()),
])
# tune pipeline step parameters with the 'step__param' naming:
param_grid = {"nb__alpha": [0.01, 0.1, 1.0], "nb__fit_prior": [True, False]}
```

#	Optimization	Blind spot it fixes	Tool
1	Log space	Underflow to zero	Sums of logs (built in)
2	Drop correlated features	Double-counting evidence	Correlation check, feature selection
3	Text pre-processing	Useless words add noise	stop_words, custom word lists

#	Optimization	Blind spot it fixes	Tool
4	Normalize	Long texts dominate short ones	TF / TF-IDF, norm='l2'
5	Parallelize	Slow on big data	n_jobs=-1

PART 7

Grid Search in Practice

Pro: putting it all together

What is Grid Search?

Grid Search tries **every combination** of the hyperparameter values you list, scores each one with cross-validation, and keeps the best. Two alpha values and two fit_prior values means 2 × 2 = 4 combinations; with 5-fold cross-validation that is 20 model fits.

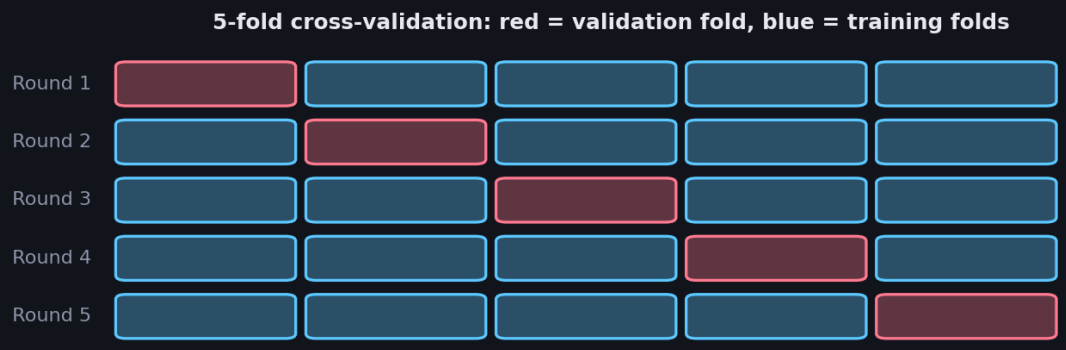


Figure 13. Each round holds out a different fold for validation, so every row of data gets validated exactly once.

Ready-to-use Grid Search dictionaries

If you set up GridSearchCV in Python, copy the grid for the variation you are tuning.

For MultinomialNB:

```
param_grid = {
    'alpha': [0.01, 0.1, 0.5, 1.0, 10.0],
    'fit_prior': [True, False]
}
```

For BernoulliNB:

```
param_grid = {
    'alpha': [0.01, 0.1, 0.5, 1.0, 10.0],
    'fit_prior': [True, False],
    'binarize': [None, 0.0, 0.5]
}
```

For GaussianNB:

```
import numpy as np

param_grid = {
    'var_smoothing': np.logspace(0, -9, num=10)
}
```

Full template: MultinomialNB + GridSearchCV

This template uses MultinomialNB (the standard choice for text or discrete count data) and GridSearchCV to automatically test and evaluate hyperparameter combinations using 5-fold cross-validation.

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

# 1. Load and prepare data
# Replace with your actual dataset. X must be numeric features.
# If using text, ensure it is passed through a Vectorizer first.
X = ...
y = ...

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the base Naive Bayes model
nb_model = MultinomialNB()

# 3. Define the hyperparameter grid
# Testing a logarithmic scale for alpha to find the right smoothing balance
param_grid = {
    'alpha': [0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0],
    'fit_prior': [True, False]
}

# 4. Configure GridSearchCV
# cv=5: Splits training data into 5 folds for robust validation
# n_jobs=-1: Utilizes all available CPU cores for parallel processing
grid_search = GridSearchCV(
    estimator=nb_model,
    param_grid=param_grid,
    scoring='accuracy',  # Swap to 'f1_macro' if your classes are highly imbalanced
    cv=5,
    n_jobs=-1,
    verbose=1
)

# 5. Execute the tuning process
print("Starting Grid Search...")
grid_search.fit(X_train, y_train)

# 6. Extract the best model and its configuration
best_model = grid_search.best_estimator_

print("\n--- Tuning Results ---")
print(f"Best Hyperparameters: {grid_search.best_params_}")
print(f"Best Cross-Validation Score: {grid_search.best_score_:.4f}")

# 7. Evaluate the optimized model on the unseen test set
predictions = best_model.predict(X_test)
print("\n--- Final Test Set Evaluation ---")
print(classification_report(y_test, predictions))
```

The template, step by step

Step	What happens	Why it matters
1. Load data	X = features, y = labels. Text must go through a Vectorizer first	Naive Bayes needs numbers, not raw strings
Split 80/20	test_size=0.2, random_state=42	The 20% test set stays untouched until the very end, giving an honest score. random_state makes it repeatable
2. Base model	MultinomialNB()	The blank model whose settings will be tuned
3. Grid	alpha on a log scale, fit_prior True/False	7 × 2 = 14 combinations

Step	What happens	Why it matters
4. GridSearchCV	cv=5, n_jobs=-1, scoring, verbose=1	cv=5: robust validation. n_jobs=-1: all CPU cores (Optimization 5). scoring='accuracy' or 'f1_macro' for imbalanced classes. verbose=1: progress messages
5. Fit	grid_search.fit(X_train, y_train)	14 combinations × 5 folds = 70 fits, run in parallel
6. Best model	best_estimator_, best_params_, best_score_	The winning settings and its average cross-validation score
7. Final test	predict + classification_report	Precision, recall and F1 on unseen data

Accuracy or f1_macro?
Accuracy can look great while ignoring a rare class (a model that always says "Not Spam" on 95% non-spam data scores 95%). With imbalanced classes use `scoring='f1_macro'`, which averages F1 equally over all classes.

Pro checklist and common mistakes

- Always tune on a log scale for alpha and var_smoothing.
- Only use the tuned hyperparameters that exist for your variant (no binarize in MultinomialNB, no alpha in GaussianNB).
- Never tune on the test set. The test set is used once, at the end.
- Check the best value is not at the edge of your grid; if it is, extend the grid in that direction and search again.
- Compare against the defaults (alpha=1.0, var_smoothing=1e-9) to make sure tuning really helped.
- Apply the five optimizations (Part 6) before tuning: clean features beat clever hyperparameters.

One-page cheat sheet

Hyperparameter	Variant	Low value does...	High value does...	Default	Search values
alpha	Multinomial, Bernoulli, Categorical	Trusts data; overfits (high variance)	Flattens everything; underfits (high bias)	1.0	[0.001 ... 100] log scale
fit_prior	Multinomial, Bernoulli, Categorical	False: equal class chance	True: learn class frequencies	True	[True, False]
binarize	Bernoulli	Low threshold: more 1s	High threshold: fewer 1s	0.0	[None, 0.0, 0.5, 1.0]
var_smoothing	Gaussian	Narrow curves; unstable	Wide flat curves; underfits	1e-9	[1e-11 ... 1e-5] log scale

You made it

You now know why Naive Bayes multiplies probabilities, what can break (zeros, tiny numbers, narrow curves, double-counting), and exactly which hyperparameter fixes each problem. Next step: run the template on your own data, then try the TF-IDF pipeline.